

Filipino Cookbook API

A. API Description

The Filipino Cookbook API is a RESTful web service developed using the Slim Framework and PHP. It provides access to information about Filipino dishes, including their categories, origins, ingredients, and cooking instructions. The API allows users to retrieve, create, update, and delete food records while returning responses in JSON format. It is intended for students, developers, and client applications that require structured Filipino recipe data. The project also demonstrates secure API development through authentication, validation, rate limiting, and proper error handling.

B. Objective

The objective of this activity is to design, develop, and test a secure RESTful API for the Filipino Cookbook System using the Slim PHP Framework. The API enables users to perform CRUD (Create, Read, Update, Delete) operations on Filipino food records while implementing essential security features such as Bearer Token Authentication, Input Validation, Rate Limiting, and secure error handling to ensure reliable and protected API communication. Additionally, the activity demonstrates proper handling of successful requests, validation errors, unauthorized access, missing resources, and other exceptions by returning appropriate HTTP status codes and JSON responses. All API endpoints are tested using Thunder Client to verify their functionality, accuracy, and security.

C. FEATURES:

The following table summarizes the key features implemented in the Filipino Cookbook API, along with a brief description of each functionality.

Feature	Description
1. Retrieve All Filipino Foods	Displays all Filipino food records stored in the database.
2. Retrieve Food by ID	Returns the complete details of a specific Filipino food.
3. Search Food by Name	Searches for Filipino foods using the provided food name.
4. Retrieve Random Food	Returns a randomly selected Filipino food from the database.
5. Retrieve Categories	Displays all available food categories.
6. Retrieve Ingredients	Displays all available ingredients.
7. Add New Food	Creates a new Filipino food record with its corresponding details and ingredients.
8. Update Existing Food	Updates the information of an existing Filipino food record.
9. Delete Food	Removes a Filipino food record from the database.
10. Bearer Token Authentication	Secures protected API endpoints by requiring a valid Bearer Token.
11. Input Validation	Checks user input to ensure that submitted data is complete and valid.
12. Rate Limiting	Restricts excessive API requests to prevent abuse and improve security.
13. Secure Error Handling	Returns appropriate error messages and HTTP status codes without exposing sensitive system information.
14. JSON Response Format	Returns all API responses in a consistent JSON format for easy integration.

D. TECHNOLOGY USED

The development of the Filipino Cookbook API utilized various software tools, frameworks, and technologies. The following table lists each technology and its purpose in the development, testing, and deployment of the project.

Technology / Tool	Purpose
Windows 10/11	Operating system used during development
Visual Studio Code (VS Code)	Source code editor used to develop and edit the API
XAMPP	Local development environment that provides Apache and MySQL services
Apache Web Server	Web server used to host and run the API locally
PHP 8.x	Server-side programming language used to develop the RESTful API
Slim Framework 4	PHP micro-framework used to build the RESTful API
Composer	Dependency manager used to install and manage PHP packages
MySQL	Relational Database Management System used to store API data
phpMyAdmin	Web-based database management tool for MySQL
JSON	Data format used for API requests and responses
Thunder Client (VS Code Extension)	API testing tool used to send and validate HTTP requests and responses
Git	Version control system used to track source code changes
GitHub	Repository hosting platform used for version control and project documentation

E. INSTALLATION INSTRUCTION

Note: If you already have the required software and tools installed on your computer, you may skip this section and proceed to Project Setup and Configuration.

Step 1: Install Visual Studio Code

- Visit the official Visual Studio Code website:
<https://code.visualstudio.com/>
- Download the installer for your operating system.
- Run the installer and follow the installation wizard.
- Launch Visual Studio Code after installation.

Step 2: Install XAMPP

- Download XAMPP from the official Apache Friends website:
<https://www.apachefriends.org/>
- Install XAMPP using the default settings.
- Open the XAMPP Control Panel.
- Start the following services:
 1. Apache
 2. MySQL
- Confirm that Apache is working by opening the following address in a browser:

<http://localhost>

- Confirm that phpMyAdmin is accessible through:

<http://localhost/phpmyadmin>

Note: The project folder should be placed inside the XAMPP web directory

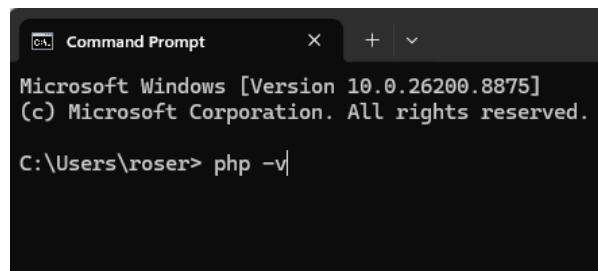
<C:\xampp\htdocs>

Step 3: Install PHP

PHP is already included with XAMPP.

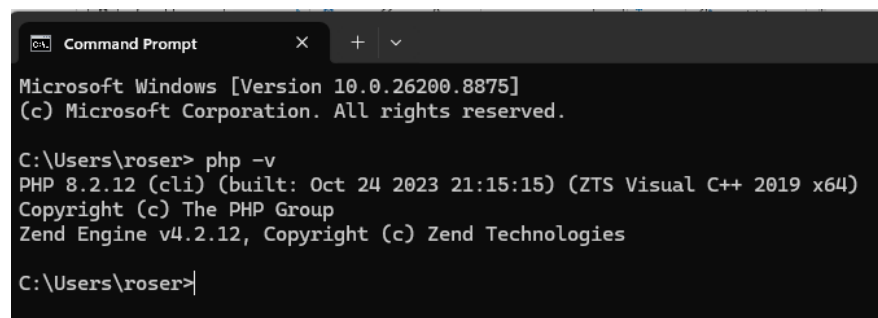
To verify the installation:

1. Open Command Prompt.
2. Type:



```
Command Prompt
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.
C:\Users\roser> php -v|
```

Note: If the PHP version is displayed, PHP has been installed successfully.



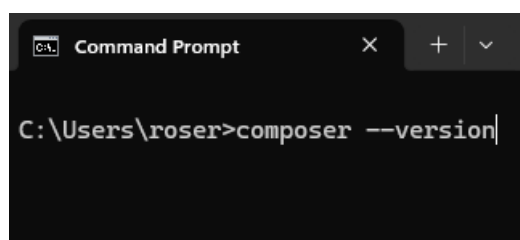
```
Command Prompt
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.
C:\Users\roser> php -v
PHP 8.2.12 (cli) (built: Oct 24 2023 21:15:15) (ZTS Visual C++ 2019 x64)
Copyright (c) The PHP Group
Zend Engine v4.2.12, Copyright (c) Zend Technologies
C:\Users\roser>
```

Step 4: Install Composer

- Download Composer from: <https://getcomposer.org/>
- Run the Composer installer.
- Click the Download button.
- Under Windows Installer, click Composer-Setup.exe.
- During installation, browse to the PHP executable located inside XAMPP, Select this:

C:\xampp\php\php.exe

- Complete the installation.
- Verify the installation:



```
Command Prompt
C:\Users\roser>composer --version|
```

Note: If the composer version is displayed, composer has been installed successfully.

```
Command Prompt

C:\Users\roser>composer --version
Composer version 2.10.2 2026-07-01 11:24:45
PHP version 8.2.12 (C:\xampp\php\php.exe)
Run the "diagnose" command to get more detailed diagnostics output.

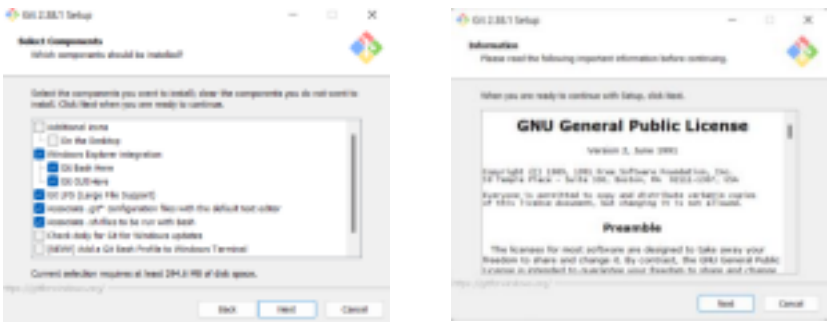
C:\Users\roser>
```

Step 5: Install Git

- Download Git from: <https://git-scm.com/>



- Install Git using the recommended default settings.



- Verify the installation:

```
Command Prompt

C:\Users\roser>git --version
```

Note: If the git version is displayed, git has been installed successfully.

```
Command Prompt

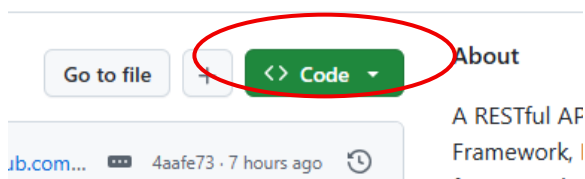
C:\Users\roser>git --version
git version 2.55.0.windows.3

C:\Users\roser>
```

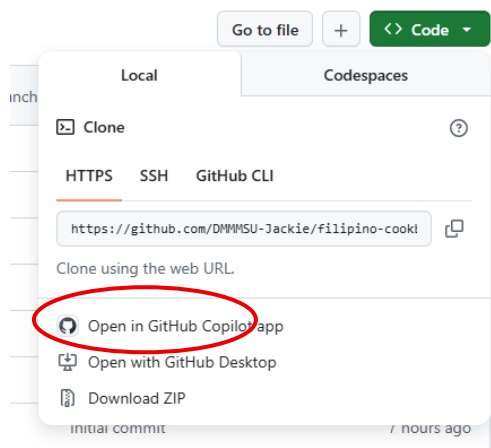
F. PROJECT SET-UP

Step 1: Method A - Download the Repository as a ZIP File

- Open the Filipino Cookbook API GitHub repository.
- Select the green Code button.



- Select Download ZIP.



- Wait for the file to finish downloading. The file will usually be saved in the computer's **Downloads** folder.
- Extract the ZIP file. Choose the location where the extracted file will be saved:

C:\xampp\htdocs

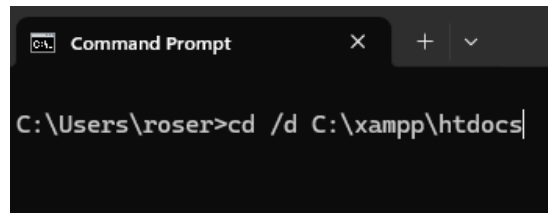
- Select **Extract**.

The extracted folder may initially have a name similar to:

“filipino-cookbook-api-maqui-main”

Method B - Clone the Repository Using Git (This method is recommended for users who want to download future updates using Git.)

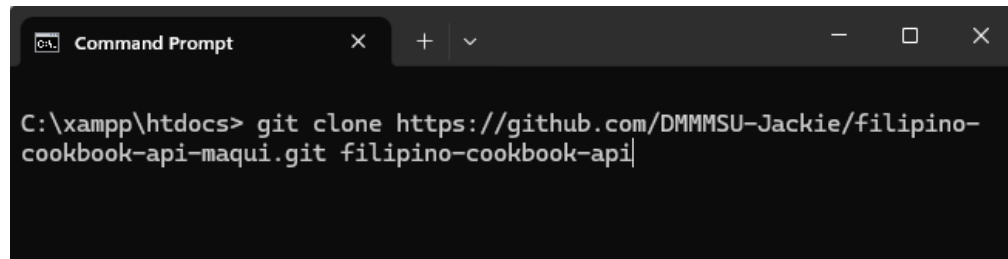
- Open Command Prompt



```
C:\Users\roser>cd /d C:\xampp\htdocs\
```

This command changes the current directory to the XAMPP web directory.

- Clone the GitHub repository

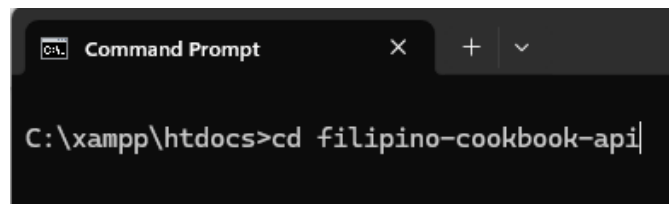


```
C:\xampp\htdocs> git clone https://github.com/DMMMSU-Jackie/filipino-cookbook-api-maqui.git filipino-cookbook-api\
```

This command:

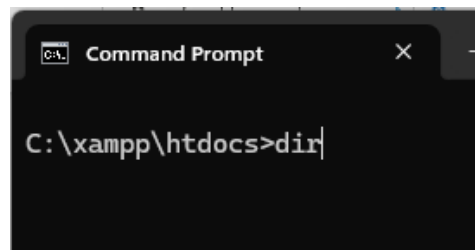
- Connects to the GitHub repository.
- Downloads all project files.
- Creates a local folder named filipino-cookbook-api.

- Enter the project folder



```
C:\xampp\htdocs>cd filipino-cookbook-api\
```

- Verify the downloaded files:



```
C:\xampp\htdocs>dir\
```

- The output should include files or folders such as:

config
middleware
public
routes
storage
vendor
README.md
composer.json
composer.lock

NOTE: The exact structure may differ slightly depending on the repository.

Step 2: Open the Project and Configure the VS Code Terminal

- Open Visual Studio Code.
- Select File, then Open Folder.
- Choose C:\xampp\htdocs\filipino-cookbook-api-maqui.
- Select Terminal, then New Terminal, or press Ctrl + backtick (`).
- Confirm that the terminal path points to the project folder

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\xampp\htdocs\filipino-cookbook-api> dir

Directory: C:\xampp\htdocs\filipino-cookbook-api

Mode                LastWriteTime         Length Name
----                -
d-----          12/07/2026   7:42 pm             config
d-----          12/07/2026   7:44 pm             middleware
d-----          12/07/2026   8:00 pm             public
d-----          13/07/2026   4:36 am             routes
d-----          22/07/2026   2:47 pm             storage
d-----          12/07/2026   7:39 pm             vendor
-a----          25/07/2026   7:25 pm              83 composer.json
-a----          12/07/2026   7:27 pm          24746 composer.lock
-a----          22/07/2026   4:08 pm           239 README.md
```

Step 3: Restore and Configure Slim Framework through Composer

- Because the API is already uploaded to GitHub, the correct command is `composer install`. This command reads `composer.json` and `composer.lock`, downloads Slim Framework and its supporting packages, creates the vendor folder, and generates `vendor/autoload.php`.

```
composer install
```

- **Do not normally run these commands for the downloaded project:**

```
composer init  
composer require slim/slim:"4.*"  
composer require slim/psr7
```

- Those commands are used when creating a new Slim project or when the dependency entries are missing. For the downloaded repository, `composer install` is the reproducible installation step.

- **Verify that Slim was restored:**

```
composer show slim/slim  
composer show slim/psr7  
composer show
```

- A successful result displays `slim/slim`, `slim/psr7`, and the related PSR packages. The project should now contain a vendor folder and `vendor/autoload.php`.

Step 4: Fix Slim Framework or Composer Installation Problems

➤ Problem A — composer.json is missing

This usually means the terminal is open in the wrong folder or the repository was extracted incorrectly. Use `dir` to locate `composer.json`, then change to the correct directory before running Composer.

```
dir
cd /d C:\xampp\htdocs\filipino-cookbook-api-maqui
```

➤ Problem B — vendor/autoload.php is missing

```
composer install
composer dump-autoload
```

➤ Problem C — Slim packages are not listed in composer.json

Use this only when the repository truly does not declare Slim. This changes `composer.json` and `composer.lock`, so the developer should commit the corrected dependency files afterward.

```
composer require slim/slim:"4.*"
composer require slim/psr7
composer dump-autoload
```

➤ Problem D — PHP extension is missing

Open `C:\xampp\php\php.ini` and enable the required extension by removing the semicolon at the beginning of the line. Common requirements include `pdo_mysql` and `mbstring`. Save the file, restart Apache or reopen the terminal, and run `composer install` again.

```
extension=pdo_mysql
extension=mbstring

composer check-platform-reqs
```

➤ Problem E — package versions are incompatible

Check the PHP version and platform requirements. Do not immediately use `composer update` because it can replace the tested versions recorded in `composer.lock`.

```
php -v
composer check-platform-reqs
composer diagnose
```

Step 5: Confirm the Slim Entry Point

- Open public/index.php and confirm that it loads Composer's autoloader and creates the Slim application. The exact project code should remain unchanged; the following lines show the essential configuration that must be present.

```
<?php
use Slim\Factory\AppFactory;

require __DIR__ . '/../vendor/autoload.php';

$app = AppFactory::create();
$app->addBodyParsingMiddleware();
$app->addRoutingMiddleware();
$app->addErrorMiddleware(false, true, true);

// Routes and middleware are loaded here.
$app->run();
```

Step 6: Create and Import the Database

- Start MySQL from XAMPP Control Panel.
- Open <http://localhost/phpmyadmin>.
- Select New and create a database named `filipino_cookbook_api`.
- Select the new database, open the Import tab, and choose the .sql file included in the repository.
- Keep the default import options and select Go.
- Confirm that the expected tables and sample records were created.

Step 7: Configure the Database Connection

- Locate the database configuration file included in the repository. Depending on the project structure, it may be inside config, src, or another configuration folder. Enter the local MySQL values shown below.

```
Host: localhost
Database: filipino_cookbook_api
```

```
Username: root
Password: [blank for a default XAMPP installation]
```

- Do not publish real production credentials. The repository should contain placeholder values or a config.example.php file, while the private configuration file should be excluded through .gitignore.

Step 8: Run the API

- Option A — PHP Built-in Server

```
cd /d C:\xampp\htdocs\filipino-cookbook-api-maqui
php -S localhost:8080 -t public
```

Keep the terminal open. The base URL is http://localhost:8080/api.

- Option B — Apache through XAMPP

Start Apache and MySQL in XAMPP, then use the project path below. This option depends on the Apache rewrite and public-folder configuration included in the repository.

```
http://localhost/filipino-cookbook-api-maqui/public/api
```

Step 9: Test the Installation

- Open Thunder Client in Visual Studio Code.
- Create a new request.
- Use GET and enter http://localhost:8080/api/foods.
- Add Authorization: Bearer dmmmsu-cookbook-token-2026.
- Add Accept: application/json.
- Select Send and confirm that the response contains valid JSON and HTTP 200.

G. DATABASE SETUP

Item	Details
Database name	filipino_cookbook_api
SQL file	[Enter the exact .sql filename included in the repository.]
Database server	MySQL through XAMPP
Administration tool	phpMyAdmin
Primary data	Foods, categories, ingredients, and their relationships

Expected Table Relationships

```
categories → foods
foods → food_ingredients ← ingredients
```

- A category may be assigned to multiple foods. A food may use multiple ingredients, and an ingredient may appear in multiple foods. The food_ingredients junction table represents the many-to-many relationship between foods and ingredients. The final documentation must use the exact table and foreign-key names found in the included SQL file.

H. BASE URL

Environment	Base URL
PHP built-in server	http://localhost:8080/api
XAMPP/Apache	http://localhost/filipino-cookbook-api-maqui/public/api

- Use only one base URL consistently during testing. The endpoint paths in the following section are appended to the selected base URL.

I. AUTHENTICATION INSTRUCTIONS

- The API uses Bearer Token Authentication. Protected requests must include the Authorization header exactly as shown below.

```
Authorization: Bearer dmmsu-cookbook-token-2026
Accept: application/json
Content-Type: application/json
```

- Content-Type is required for requests containing a JSON body, such as POST and PUT. When the token is missing or invalid, the API should return HTTP 401 with a JSON error response. The public repository should not expose a real private production token; use a placeholder or a documented classroom token only.

J. ENDPOINT DOCUMENTATION

This section documents every available endpoint implemented in the Filipino Cookbook API. Each endpoint includes its purpose, request method, URL, required headers, sample request, response placeholders, and a placeholder where Thunder Client screenshots can be inserted.

1. API Root

Method	GET
Endpoint	http://localhost:8080/
Description	Returns the welcome message confirming that the API server is running.
Headers	Accept: application/json
Example Request	GET http://localhost:8080/

Successful Response

```
{
  "message": "Welcome to the Secured Filipino Cookbook API",
  "note": "Use a valid Bearer token to access /api endpoints."
}
```

Error Response

No error response because this can be access without an authorization

2. Retrieve All Foods

Method	GET
Endpoint	http://localhost:8080/api/foods
Description	Returns all Filipino food records.
Headers	Authorization: Bearer dmmmsu-cookbook-token-2026 Accept: application/json
Example Request	GET http://localhost:8080/api/foods

Successful Response

```
[
  {
    "food_id": 1,
```

```
"food_name": "Adobo",

"category_name": "Main Dish",

"origin_name": "Philippines",

"instructions": "Marinate the meat with soy sauce, vinegar, garlic, bay
leaves, and peppercorn. Simmer until the meat becomes tender and the
sauce is reduced.",

"ingredients": [

    "Bay leaves",

    "Chicken or pork",

    "Cooking oil",

    "Garlic",

    "Peppercorn",

    "Soy sauce",

    "Vinegar"

] .....
```

Error Response

```
[

    {

        "status": "error",

        "message": "Unauthorized access. Valid API token is required."

    }

]
```

3. Retrieve Food by ID

Method	GET
Endpoint	http://localhost:8080/api/foods/(food_id)
Description	Returns the details of the specified food.

Headers	Authorization: Bearer dmmmsu-cookbook-token-2026 Accept: application/json
Example Request	GET http://localhost:8080/api/foods/2

Successful Response

```
{
  "food_id": 2,
  "food_name": "Sinigang",
  "category_name": "Soup",
  "origin_name": "Philippines",
  "instructions": "Boil the meat or seafood with tomatoes. Add tamarind mix
and vegetables, then simmer until cooked.",
  "ingredients": [
    "Green chili",
    "Kangkong",
    "Pork, shrimp, or fish",
    "Radish",
    "String beans",
    "Tamarind mix",
    "Tomato"
  ]
}
```

Error Response

```
{
  "status": "error",
  "message": "Food not found"
}
```

4. Search Food by Name

Method	GET
Endpoint	http://localhost:8080/api/foods/search/(food_name)
Description	Searches for foods matching the given name.
Headers	Authorization: Bearer dmmmsu-cookbook-token-2026 Accept: application/json
Example Request	GET http://localhost:8080/api/foods/search/adobo

Successful Response

```
[
  {
    "food_id": 1,
    "food_name": "Adobo",
    "category_name": "Main Dish",
    "origin_name": "Philippines",
    "instructions": "Marinate the meat with soy sauce, vinegar, garlic, bay leaves, and peppercorn. Simmer until the meat becomes tender and the sauce is reduced."
  }
]
```

Error Response

```
{
  "status": "error",
  "message": "Food not found."
}
```

5. Retrieve Categories

Method	GET
Endpoint	http://localhost:8080/api/categories
Description	Returns all food categories.
Headers	Authorization: Bearer dmmmsu-cookbook-token-2026 Accept: application/json
Example Request	GET http://localhost:8080/api/categories

Successful Response

```
[
  {
    "category_id": 1,
    "category_name": "Appetizer"
  },
  {
    "category_id": 2,
    "category_name": "Dessert"
  },
  {
    "category_id": 3,
    "category_name": "Grilled Dish"
  },
  {
    "category_id": 4,
    "category_name": "Main Dish"
  }
]
```

```
    "category_id": 5,

    "category_name": "Noodle Dish"

},

{

    "category_id": 6,

    "category_name": "Soup"

},

{

    "category_id": 7,

    "category_name": "Vegetable Dish"

}

]
```

Error Response

```
{

    "message": "404 Not Found",

    "exception": [

        {

            "type": "Slim\\Exception\\HttpNotFoundException",

            "code": 404,

            "message": "Not found.",

            "file": "C:\\xampp\\htdocs\\filipino-cookbook-

api\\vendor\\slim\\slim\\Slim\\Middleware\\RoutingMiddleware.php",

            "line": 76

        }

    ]

}
```

6. Retrieve Ingredients

Method	GET
Endpoint	http://localhost:8080/api/ingredients
Description	Returns all ingredients.
Headers	Authorization: Bearer dmmmsu-cookbook-token-2026 Accept: application/json
Example Request	GET http://localhost:8080/api/ingredients

Successful Response

```
[
  {
    "ingredient_id": 1,
    "ingredient_name": "Annatto oil"
  },
  {
    "ingredient_id": 2,
    "ingredient_name": "Bagoong"
  },
  {
    "ingredient_id": 3,
    "ingredient_name": "Banana blossom"
  },
  {
    "ingredient_id": 4,
    "ingredient_name": "Bay leaves"
  },
  {.....
```

Error Response

```
{
  "message": "404 Not Found",
  "exception": [
    {
      "type": "Slim\\Exception\\HttpNotFoundException",
      "code": 404,
      "message": "Not found.",
      "file": "C:\\xampp\\htdocs\\filipino-cookbook-
api\\vendor\\slim\\slim\\Slim\\Middleware\\RoutingMiddleware.php",
      "line": 76
    }
  ]
}
```

7. Add New Food

Method	POST
Endpoint	http://localhost:8080/api/foods
Description	Creates a new food record.
Headers	Authorization: Bearer dmmmsu-cookbook- token-2026 Accept: application/json
Example Request	POST http://localhost:8080/api/foods
Request Body	{ "name": "...", "category_id":1, "origin_id":1, "instructions":"...", "ingredients":[1,2] }

Sample Request Body	<pre>{ "food_name": "Dinengdeng", "category_id": 3, "origin_id": 4, "instructions": "Boil vegetables with bagoong-based broth and add grilled fish before serving.", "ingredient_ids": [10, 15, 22] }</pre>
---------------------	---

Successful Response

```
{
  "status": "success",
  "message": "Food added successfully."
}
```

Error Response

```
{
  "status": "error",
  "message": "Food 'Adobo' already exists."
}
```

8. Random Food

Method	GET
Endpoint	http://localhost:8080/api/foods/random
Description	Returns one random Filipino food.
Headers	Authorization: Bearer dmmmsu-cookbook- token-2026 Accept: application/json
Example Request	GET http://localhost:8080/api/foods/random

Successful Response

```
{
  "food_id": 2,
  "food_name": "Sinigang",
  "category_name": "Soup",
  "origin_name": "Philippines",
  "instructions": "Boil the meat or seafood with tomatoes. Add tamarind mix
and vegetables, then simmer until cooked.",
  "ingredients": [
    "Green chili",
    "Kangkong",
    "Pork, shrimp, or fish",
    "Radish",
    "String beans",
    "Tamarind mix",
    "Tomato"
  ]
}
```

Error Response

```
{
  "message": "405 Method Not Allowed",
  "exception": [
    {
      "type": "Slim\\Exception\\HttpMethodNotAllowedException",
      "code": 405,
      "message": "Method not allowed. Must be one of: GET",
      "file": "C:\\xampp\\htdocs\\filipino-cookbook-
api\\vendor\\slim\\slim\\Slim\\Middleware\\RoutingMiddleware.php",
```



```
"line": 79

}

]

}
```

9. Delete Food

Method	DELETE
Endpoint	http://localhost:8080/api/foods/16
Description	Deletes the specified food record.
Headers	Authorization: Bearer dmmmsu-cookbook-token-2026 Accept: application/json
Example Request	DELETE http://localhost:8080/api/foods/16

Successful Response

```
{

  "status": "success",

  "message": "Food deleted successfully."

}
```

Error Response

```
{

  "status": "error",

  "message": "Food not found"

}
```

10. Update Food

Method	PUT
Endpoint	http://localhost:8080/api/foods/16
Description	Updates the specified food record.

Headers	Authorization: Bearer dmmmsu-cookbook-token-2026 Accept: application/json
Example Request	PUT http://localhost:8080/api/foods/16
Request Body	{ "name":"...", "category_id":1, "origin_id":1, "instructions":"...", "ingredients":[1,2] }

Successful Response

```
{  
  "status": "success",  
  "message": "Food updated successfully."  
}
```

Error Response

```
{  
  "status": "error",  
  "message": "Invalid or incomplete food data."  
}
```

K. TESTING EVIDENCE

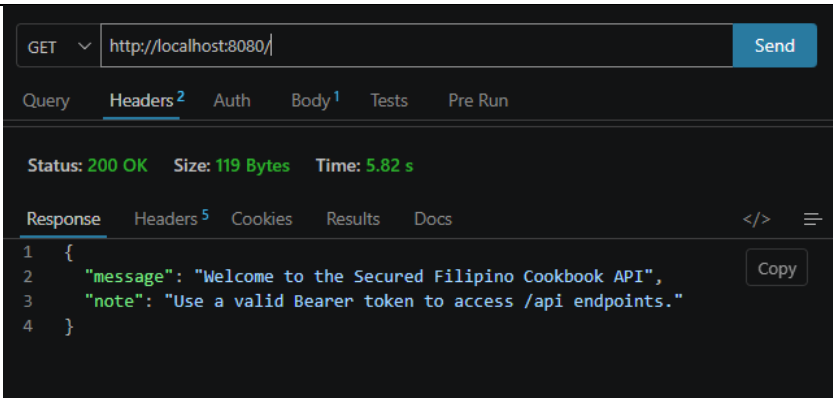
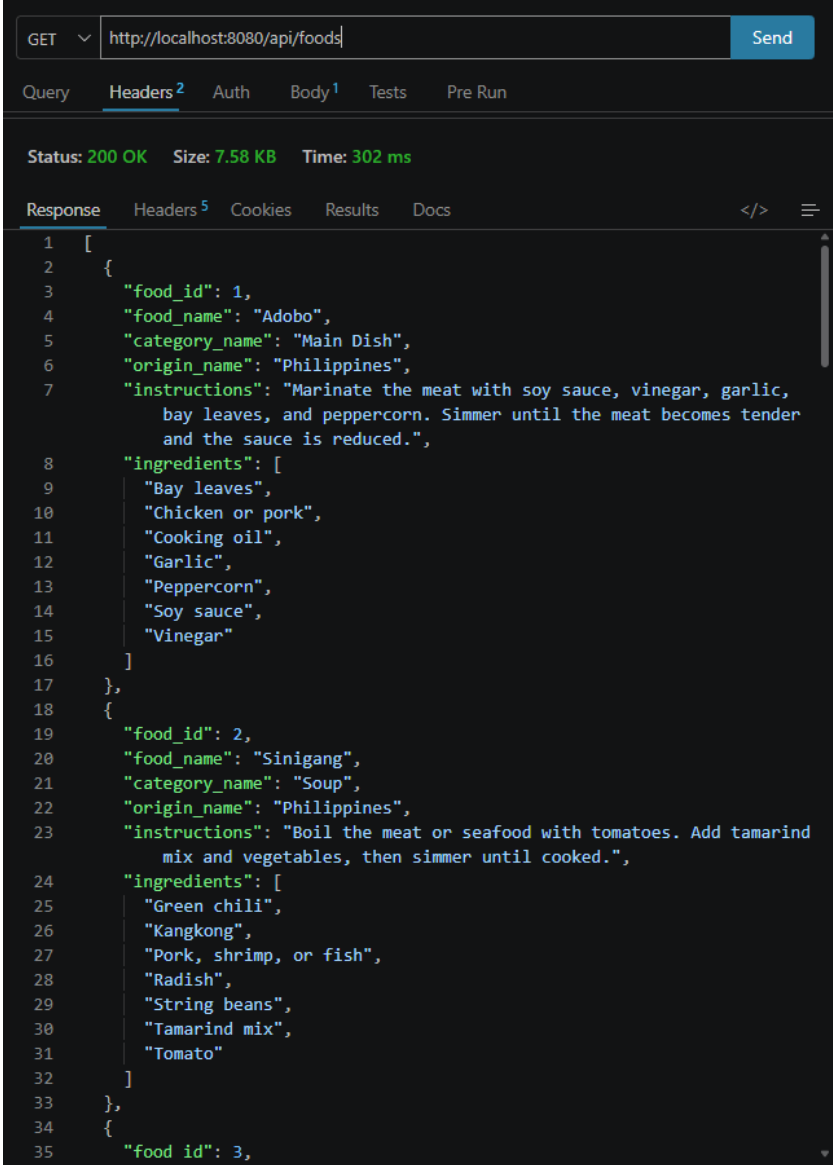
Figure	Description	Screenshot
Figure 1	API Root	 <pre>GET http://localhost:8080/ Status: 200 OK Size: 119 Bytes Time: 5.82 s Response Headers Cookies Results Docs 1 { 2 "message": "Welcome to the Secured Filipino Cookbook API", 3 "note": "Use a valid Bearer token to access /api endpoints." 4 }</pre>
Figure 2	Retrieve All Foods	 <pre>GET http://localhost:8080/api/foods Status: 200 OK Size: 7.58 KB Time: 302 ms Response Headers Cookies Results Docs 1 [2 { 3 "food_id": 1, 4 "food_name": "Adobo", 5 "category_name": "Main Dish", 6 "origin_name": "Philippines", 7 "instructions": "Marinate the meat with soy sauce, vinegar, garlic, 8 bay leaves, and peppercorn. Simmer until the meat becomes tender 9 and the sauce is reduced.", 10 "ingredients": [11 "Bay leaves", 12 "Chicken or pork", 13 "Cooking oil", 14 "Garlic", 15 "Peppercorn", 16 "Soy sauce", 17 "Vinegar" 18] 19 }, 20 { 21 "food_id": 2, 22 "food_name": "Sinigang", 23 "category_name": "Soup", 24 "origin_name": "Philippines", 25 "instructions": "Boil the meat or seafood with tomatoes. Add tamarind 26 mix and vegetables, then simmer until cooked.", 27 "ingredients": [28 "Green chili", 29 "Kangkong", 30 "Pork, shrimp, or fish", 31 "Radish", 32 "String beans", 33 "Tamarind mix", 34 "Tomato" 35] 36 }, 37 { 38 "food_id": 3,</pre>

Figure 3	Retrieve Food by ID	GET ⌵ http://localhost:8080/api/foods/2 Send QueryHeaders²AuthBody¹TestsPre Run Status: 200 OKSize: 421 BytesTime: 82 ms ResponseHeaders⁵CookiesResultsDocs </>≡ Copy <pre>1 { 2 "food_id": 2, 3 "food_name": "Sinigang", 4 "category_name": "Soup", 5 "origin_name": "Philippines", 6 "instructions": "Boil the meat or seafood with tomatoes. Add tamarind mix 7 and vegetables, then simmer until cooked.", 8 "ingredients": [9 "Green chili", 10 "Kangkong", 11 "Pork, shrimp, or fish", 12 "Radish", 13 "String beans", 14 "Tamarind mix", 15 "Tomato" 16] 17 }</pre>
Figure 4	Search Food by Name	GET ⌵ http://localhost:8080/api/foods/search/adobo Send QueryHeaders²AuthBody¹TestsPre Run Status: 200 OKSize: 311 BytesTime: 103 ms ResponseHeaders⁵CookiesResultsDocs </>≡ Copy <pre>1 [2 { 3 "food_id": 1, 4 "food_name": "Adobo", 5 "category_name": "Main Dish", 6 "origin_name": "Philippines", 7 "instructions": "Marinate the meat with soy sauce, vinegar, garlic, bay 8 leaves, and peppercorn. Simmer until the meat becomes tender and the 9 sauce is reduced." 10 } 11]</pre>

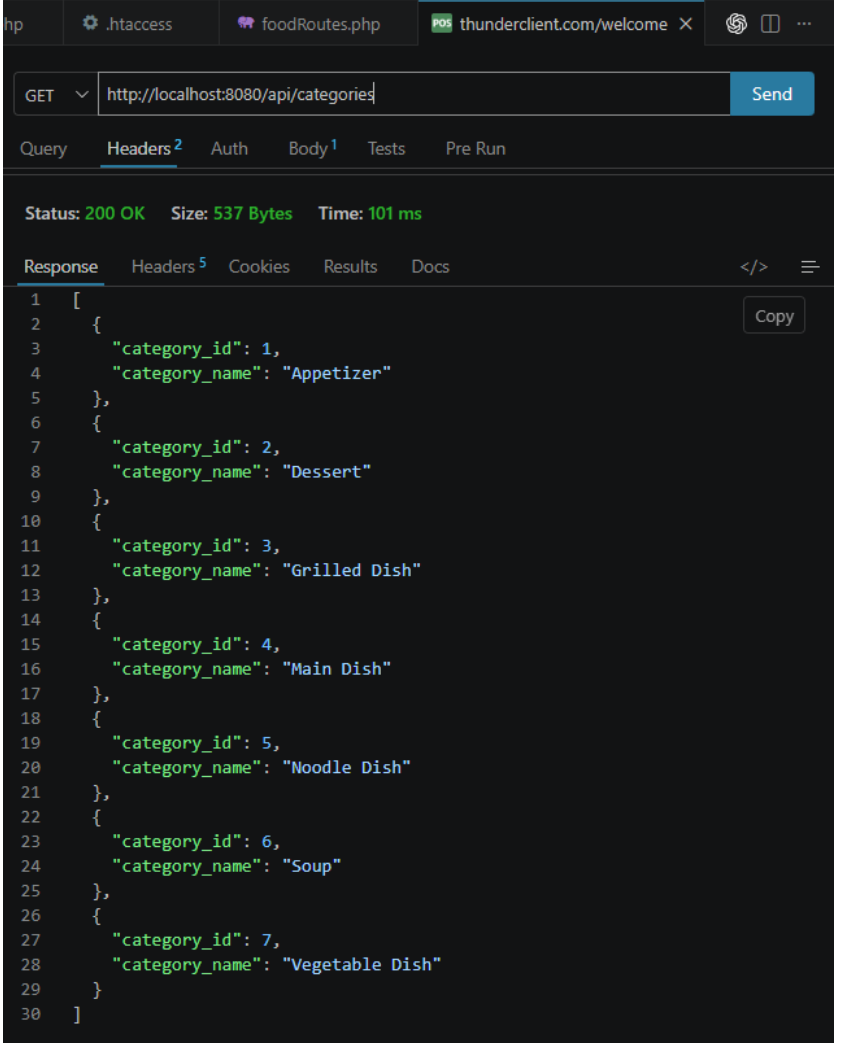
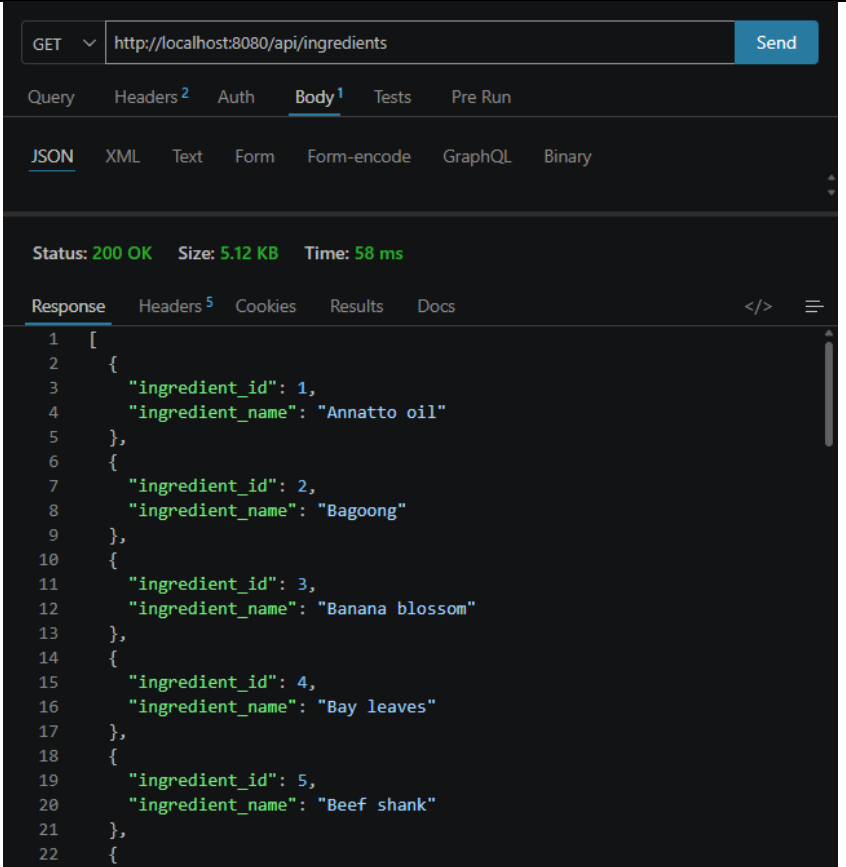
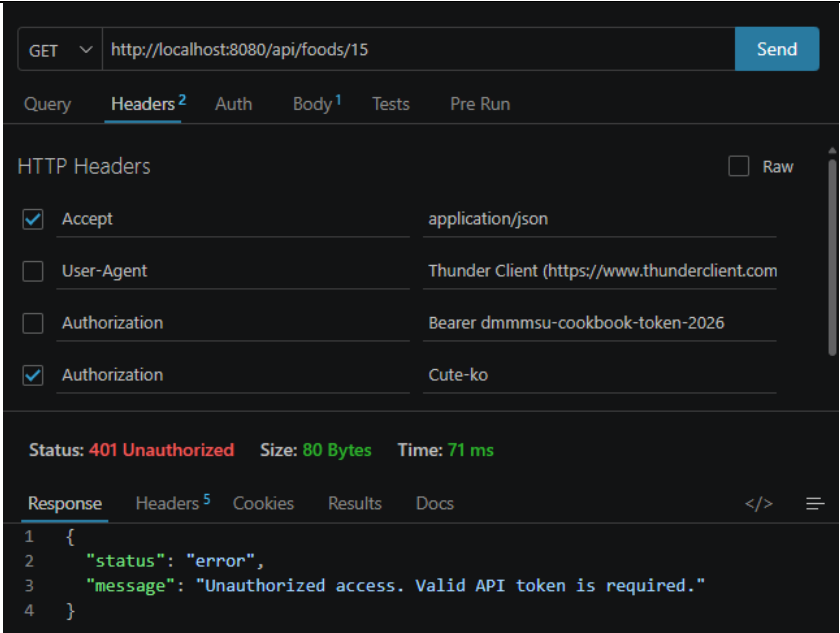
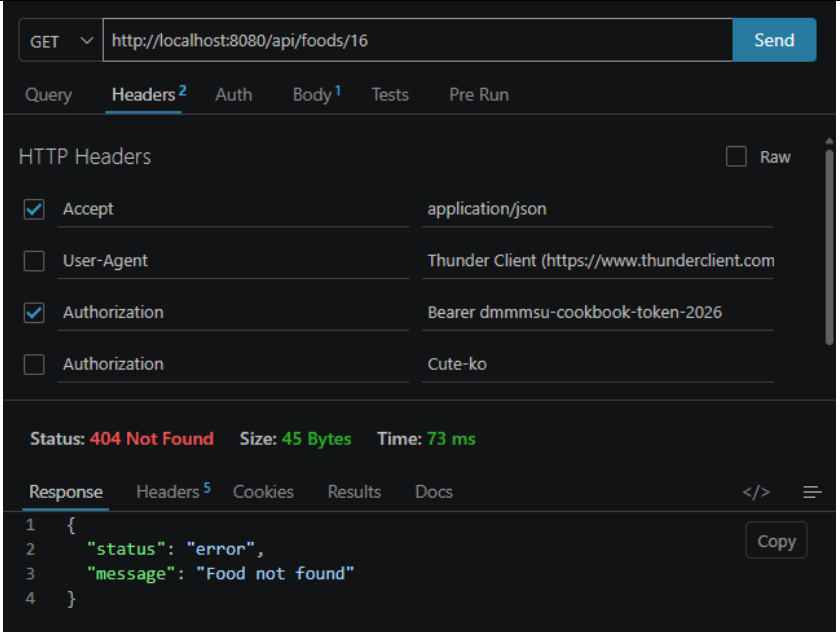
Figure 5	Retrieve Categories	 The screenshot shows the Thunderclient interface with a GET request to <code>http://localhost:8080/api/categories</code>. The response status is 200 OK, size is 537 Bytes, and time is 101 ms. The response body is a JSON array of 7 categories: <pre>[{ "category_id": 1, "category_name": "Appetizer" }, { "category_id": 2, "category_name": "Dessert" }, { "category_id": 3, "category_name": "Grilled Dish" }, { "category_id": 4, "category_name": "Main Dish" }, { "category_id": 5, "category_name": "Noodle Dish" }, { "category_id": 6, "category_name": "Soup" }, { "category_id": 7, "category_name": "Vegetable Dish" }]</pre>
Figure 6	Retrieve Ingredients	 The screenshot shows the Thunderclient interface with a GET request to <code>http://localhost:8080/api/ingredients</code>. The response status is 200 OK, size is 5.12 KB, and time is 58 ms. The response body is a JSON array of 5 ingredients: <pre>[{ "ingredient_id": 1, "ingredient_name": "Annatto oil" }, { "ingredient_id": 2, "ingredient_name": "Bagoong" }, { "ingredient_id": 3, "ingredient_name": "Banana blossom" }, { "ingredient_id": 4, "ingredient_name": "Bay leaves" }, { "ingredient_id": 5, "ingredient_name": "Beef shank" }]</pre>

Figure 7	Add New Food	POST http://localhost:8080/api/foods Send QueryHeaders 2AuthBody 1TestsPre Run JSONXMLTextFormForm-encodeGraphQLBinary JSON ContentFormat 1 2 3 4 5 6 7 "food_name": "Dinengdeng", "category_id": 3, "origin_id": 4, "instructions": "Boil vegetables with bagoong-based broth and add grilled fish before serving.", "ingredient_ids": [10, 15, 22] Status: 201 CreatedSize: 57 BytesTime: 96 ms ResponseHeaders 5CookiesResultsDocs 1 2 3 4 { "status": "success", "message": "Food added successfully." }
Figure 8	Random Food	GET http://localhost:8080/api/foods/random Send QueryHeaders 2AuthBody 1TestsPre Run JSONXMLTextFormForm-encodeGraphQLBinary JSON ContentFormat Status: 200 OKSize: 421 BytesTime: 193 ms ResponseHeaders 5CookiesResultsDocs 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 "food_id": 2, "food_name": "Sinigang", "category_name": "Soup", "origin_name": "Philippines", "instructions": "Boil the meat or seafood with tomatoes. Add tamarind mix and vegetables, then simmer until cooked.", "ingredients": ["Green chili", "Kangkong", "Pork, shrimp, or fish", "Radish", "String beans", "Tamarind mix", "Tomato"] Copy
Figure 9	Delete Food	DELETE http://localhost:8080/api/foods/16 Send QueryHeaders 2AuthBody 1TestsPre Run Status: 200 OKSize: 59 BytesTime: 39 ms ResponseHeaders 5CookiesResultsDocs 1 2 3 4 { "status": "success", "message": "Food deleted successfully." }

Figure 10	Update Food	PUT http://localhost:8080/api/foods/15 Send Query Headers² Auth Body¹ Tests Pre Run JSONXMLTextFormForm-encodeGraphQLBinary JSON ContentFormat 1 { 2 "food_name": "Dinengdeng", 3 "category_id": 3, 4 "origin_id": 4, 5 "instructions": "Boil vegetables with bagoong-based broth and add grilled fish before serving.", 6 "ingredient_ids": [10, 15, 22] 7 } Status: 200 OKSize: 59 BytesTime: 86 ms Response Headers⁵ CookiesResultsDocs 1 { 2 "status": "success", 3 "message": "Food updated successfully." 4 } Copy
-----------	-------------	--

Figure 13	Invalid Token	
Figure 14	404 Not Found	

J. HTTP Status Codes

Status Code	Standard Description	Usage in the Filipino Cookbook API
200 OK	Request completed successfully.	GET requests, successful search, successful update, successful delete, and successful random food retrieval.
201 Created	A new resource was created successfully.	POST /api/foods when a new food record is successfully added.

400 Bad Request	The request contains invalid or incomplete data.	Missing required fields, invalid JSON, empty search keyword, or invalid request parameters.
401 Unauthorized	Authentication failed.	Missing or invalid Bearer Token.
404 Not Found	The requested resource was not found.	Food ID does not exist, searched food not found, or endpoint not found.
405 Method Not Allowed	The HTTP method is not allowed for the requested endpoint.	Incorrect HTTP method used for an endpoint, such as sending POST to a GET-only endpoint.
409 Conflict	The request conflicts with existing data.	Attempting to add a food that already exists (duplicate entry).
429 Too Many Requests	Too many requests were sent within a short period.	Triggered when the API rate limit is exceeded.
500 Internal Server Error	An unexpected server or database error occurred.	Database connection failures or other unhandled server-side exceptions.